

Linear Search

L = [6, 3, 8, 10, 4, 18]

Q = 8

L = [6, 3, 8, 10, 4, 18]

↑
is this Q? i=0
No, go next

Q = 8

L = [6, 3, 8, 10, 4, 18]

↑
is this Q? i=1
No, go next

Q = 8

L = [6, 3, 8, 10, 4, 18]

↑
is this Q? i=2
Yes! Return 2

Linear Search

L = [6, 8, 3, 4, 1]

Q = 5

L = [6, 8, 3, 4, 1] i=0

↑
is this Q?
No, go next

Q = 5

L = [6, 8, 3, 4, 1] i=1

↑
is this Q?
No, go next

Q = 5

L = [6, 8, 3, 4, 1] i=2

↑
is this Q?
No, go next

Q = 5

L = [6, 8, 3, 4, 1] i=3

↑
is this Q?
No, go next

Q = 5

L = [6, 8, 3, 4, 1] i=4

↑
is this Q?
No, go next

Q = 5

L = [6, 8, 3, 4, 1]

↑
Too far, never found Q, return -1

Binary Search

L = [10, 15, 23, 33, 41] mid = low + (high - low) // 2

Q = 15

L = [10, 15, 23, 33, 41] low=0, high=4
mid = 0 + (4-0) // 2 = 2

↑
is L[mid] = Q?
No.

if L[mid] > Q:
recurse with
low = low,
high = mid - 1

if L[mid] < Q:
recurse with
low = mid + 1
high = high

↓
L = [10, 15, 23, 33, 41] low=0, high=1
mid = 0 + (1-0) // 2 = 0

↑
is L[mid] = Q?
No, low = mid + 1

L = [10, 15, 23, 33, 41] low=1, high=1
mid = 1 + (1-1) // 2 = 1

↑
is L[mid] = Q?
Yes!
return mid.

L = [10, 15, 23, 33, 41] mid = low + (high - low) // 2

Q = 33

L = [10, 15, 23, 33, 41] low=0, high=4
mid = 0 + (4-0) // 2 = 2

↑
is L[mid] = Q?
No.

if L[mid] > Q:
recurse with
low = low,
high = mid - 1

if L[mid] < Q:
recurse with
low = mid + 1
high = high

↓
L = [10, 15, 23, 33, 41] low=3, high=4
mid = 3 + (4-3) // 2 = 4

↑
is L[mid] = Q?
Yes, return mid

L = [10, 15, 23, 33, 41] mid = low + (high - low) // 2

Q = 51

L = [10, 15, 23, 33, 41] low=0, high=4
mid = 0 + (4-0) // 2 = 2

↑
is L[mid] = Q?
No.

if L[mid] > Q:
recurse with
low = low,
high = mid - 1

if L[mid] < Q:
recurse with
low = mid + 1
high = high

↓
L = [10, 15, 23, 33, 41] low=3, high=4
mid = 3 + (4-3) // 2 = 3

↑
is L[mid] = Q,
No, low = mid + 1

L = [10, 15, 23, 33, 41] low=4, high=4
mid = 4 + (4-4) // 2 = 4

↑
is L[mid] = Q,
No, low = mid + 1

↓
L = [10, 15, 23, 33, 41] low=5, high=4

low > high, Base Case
return -1. reached

Selection Sort:

L = [8 | 2 | 16 | 4 | 3]

i = 0, min = 0
j = 0

L = [8 | 2 | 16 | 4 | 3]

↑ ↑
L[min] > L[j]?
same, j → 1

L = [8 | 2 | 16 | 4 | 3]

↑ ↑
L[min] > L[j]?
Yes! min = j,
j → 2

L = [8 | 2 | 16 | 4 | 3]

↑ ↑
L[min] > L[j]?
No, j → 3

L = [8 | 2 | 16 | 4 | 3]

↑ ↑
L[min] > L[j]?
No, j → 4

L = [8 | 2 | 16 | 4 | 3]

↑ ↑
L[min] > L[j]?
No, j → 4...
End of list

↓ ↓ swap L[i]
with L[min]
L = [8 | 2 | 16 | 4 | 3]

L = [2 | 8 | 16 | 4 | 3]

Keep going until i = len(L)

Bubble Sort:

L = [8 | 2 | 16 | 4]

i = 0 Swapped = False
j = 0

L = [8 | 2 | 16 | 4]

↑ ↑
Is L[j] > L[j+1]?
Yes, swap, j → 1

L = [8 | 2 | 16 | 4]

× Swapped = True

L = [2 | 8 | 16 | 4]

↑ ↑
Is L[j] > L[j+1]?
No, j → 2

L = [2 | 8 | 16 | 4]

↑ ↑
Is L[j] > L[j+1]?
Yes, swap, j → 3

L = [2 | 8 | 16 | 4]

×

L = [2 | 8 | 4 | 16]

End of j-loop, j goes to len(L) - i - 1 = 3
i → 1

i = 1 Swapped = False
j = 0

L = [2 | 8 | 4 | 16]

↑ ↑
Is L[j] > L[j+1]?
No, j → 1

L = [2 | 8 | 4 | 16]

↑ ↑
Is L[j] > L[j+1]?
Yes, swap, j → 2

L = [2 | 8 | 4 | 16]

× Swapped = True

L = [2 | 4 | 8 | 16]

End of j-loop, j goes to len(L) - i - 1 = 2
i → 2

i = 2, j = 0, Swapped = False

L = [2 | 4 | 8 | 16] j goes to 4 - 2 - 1 = 1

↑ ↑
L[j] > L[j+1]?
No, end of j as
j goes to len(L) - i - 1
or 4 - 2 - 1 = 1

Swapped = False?

Yes, Stop loop, return list